ЛР6. Регуляризация, отбор признаков, аналитические решения — пошаговый гайд

Продолжение ЛР5: те же классы `Loss`, `Optimizer`, `LinearRegressionGD`, те же функционалы. Добавляются L_1 и L_2, расширенное признаковое пространство, сравнение с `Lasso` и `Ridge`, аналитические решения и влияние L_2 на обусловленность.

Что сдавать

Раздел	Содержание
2.1	$R_{L1} = \lambda\|w\|_1$, $R_{L2} = \frac{\lambda}{2}\|w\|_2^2$: первая и вторая производные, точки недифференцируемости — TeX в отдельных ячейках
2.2	Архитектура ЛР5 + регуляризаторы, совместимые с любым функционалом
2.3.1	≥ 3 новых признака + Пирсон; ≥ 3 линеаризующих преобразования + Пирсон снова; **исходные признаки не удалять**
2.3.2	GD + MSE + L_1: без зануления и с занулением $\|w_j\| < \varepsilon_w$; λ по валидации
2.3.3	`Lasso` sklearn; три модели: коэффициенты, RMSE, занулённые признаки
2.3.4	Анализ отбора признаков
2.4.1	Своё аналитическое МНК: $w^* = (X^T X)^{-1} X^T y$
2.4.2	Своё аналитическое Ridge: $w^* = (X^T X + \lambda I)^{-1} X^T y$, λ по валидации
2.4.3	Сравнение с `Ridge` sklearn
2.4.4	Новый оптимизатор (не из ЛР5); для каждого функционала ЛР5 — без и с L_2; три графика; RMSE

2.1. Математика регуляризаторов (в Markdown-ячейки)

Свободный член w_0 **не регуляризуется**, поэтому сумма идёт по $j = 1..d$.

$$R_{L1}(w) = \lambda \sum_{j=1}^{d} |w_j|, \qquad \frac{\partial R_{L1}}{\partial w_j} = \lambda \operatorname{sign}(w_j)\ (w_j \neq 0), \qquad \partial_{w_j} R_{L1}\big|_{w_j=0} = [-\lambda, \lambda],$$

$$\nabla R_{L1} = \lambda \operatorname{sign}(w) \text{ вне координатных гиперплоскостей}, \qquad \nabla^2 R_{L1} = 0 \text{ почти всюду}.$$

Не дифференцируем в точках, где хотя бы одна $w_j = 0$; там используется субградиент.

$$R_{L2}(w) = \frac{\lambda}{2}\|w\|_2^2, \qquad \nabla R_{L2} = \lambda w, \qquad \nabla^2 R_{L2} = \lambda I.$$

Дифференцируем всюду. Регуляризованный функционал $F = Q + R$:

$$\nabla F = \nabla Q + \nabla R, \qquad H_F = H_Q + \lambda I \ (\text{для } L_2).$$

2.2. Расширение архитектуры

Вставьте в начало ноутбука ячейки из ЛР5 с классами `Loss`, `Optimizer`, `LinearRegressionGD`, `condition_number`, `rmse`, `MSELoss`, `MAELoss`, `HuberLoss`, `LogCoshLoss`, `GD`, `Momentum`, `AdaGrad`, `RMSProp`, `Adam`. Затем добавьте регуляризаторы:

```
class Regularizer(ABC):
    name = 'reg'
    def __init__(self, lam): self.lam = lam

    @staticmethod
    def mask(w):
        m = np.ones_like(w); m[0] = 0.0     # w[0] — свободный член, не штрафуем
        return m

    @abstractmethod
    def value(self, w): ...
    @abstractmethod
    def grad(self, w): ...
    @abstractmethod
    def hessian(self, w): ...


class L1(Regularizer):
    name = 'L1'
    def value(self, w):   return self.lam * np.abs(w[1:]).sum()
    def grad(self, w):    return self.lam * np.sign(w) * self.mask(w)   # sign(0) = 0 — субградиент
    def hessian(self, w): return np.zeros((len(w), len(w)))            # 0 почти всюду


class L2(Regularizer):
    name = 'L2'
    def value(self, w):   return 0.5 * self.lam * (w[1:] @ w[1:])
    def grad(self, w):    return self.lam * w * self.mask(w)
    def hessian(self, w): return self.lam * np.diag(self.mask(w))


class RegularizedLoss(Loss):
    """F(w) = Q(w) + R(w) для любого функционала Q из ЛР5."""
    def __init__(self, loss, reg):
        self.loss, self.reg = loss, reg
        self.name = f'{loss.name} + {reg.name}(λ={reg.lam:g})'
    def value(self, X, y, w):   return self.loss.value(X, y, w) + self.reg.value(w)
    def grad(self, X, y, w):    return self.loss.grad(X, y, w) + self.reg.grad(w)
    def hessian(self, X, y, w):
        H = self.loss.hessian(X, y, w)
        return None if H is None else H + self.reg.hessian(w)
```

2.3.1. Расширенное признаковое пространство

```
import seaborn as sns
df = pd.read_csv('melb_clean.csv', parse_dates=['Date'])

# предметные признаки (≥ 3)
df['Age'] = df['Date'].dt.year - df['YearBuilt']            # возраст дома при продаже
df['AreaPerRoom'] = df['BuildingArea'] / df['Rooms']        # просторность
df['BathPerRoom'] = df['Bathroom'] / df['Rooms']            # комфорт
df['LandPerRoom'] = df['Landsize'] / df['Rooms']            # участок на комнату

base = ['Rooms', 'Distance', 'Bathroom', 'Car', 'Landsize', 'BuildingArea', 'YearBuilt',
        'Lattitude', 'Longtitude', 'Propertycount']
domain = ['Age', 'AreaPerRoom', 'BathPerRoom', 'LandPerRoom']
df['log_Price'] = np.log(df['Price'])

pear1 = df[base + domain + ['log_Price']].corr()
plt.figure(figsize=(10, 8)); sns.heatmap(pear1, cmap='coolwarm', center=0, annot=True, fmt='.2f',
                                         annot_kws={'size': 7})
plt.title('Пирсон: исходные + предметные'); plt.show()
```

Ищем нелинейные зависимости (диаграммы рассеяния `log_Price` от признаков, как в ЛР4) и добавляем ≥ 3 линеаризующих признака:

```
df['log_BA'] = np.log(df['BuildingArea'])          # степенная зависимость площади
df['log_Land'] = np.log1p(df['Landsize'])          # огромный разброс участков
df['inv_Dist'] = 1 / (df['Distance'] + 1)          # резкое падение цены у центра
df['Age_sq'] = (df['Age'] - df['Age'].mean()) ** 2  # U-образная зависимость от возраста
linear = ['log_BA', 'log_Land', 'inv_Dist', 'Age_sq']

for raw, new in [('BuildingArea', 'log_BA'), ('Landsize', 'log_Land'),
                 ('Distance', 'inv_Dist'), ('Age', 'Age_sq')]:
    print(f'{raw:13s} r={df[raw].corr(df["log_Price"]):+.3f}   {new:9s} r=
{df[new].corr(df["log_Price"]):+.3f}')

pear2 = df[base + domain + linear + ['log_Price']].corr()
plt.figure(figsize=(12, 10)); sns.heatmap(pear2, cmap='coolwarm', center=0, annot=True, fmt='.2f',
                                          annot_kws={'size': 6})
plt.title('Пирсон: расширенное пространство'); plt.show()
```

Приведение к виду для линейной модели: one-hot для категорий, разбиение, стандартизация по train.

```
df = pd.get_dummies(df, columns=['Type', 'Regionname'], drop_first=True, dtype=float)
dummies = [c for c in df.columns if c.startswith(('Type_', 'Regionname_'))]
FEATURES = base + domain + linear + dummies
X_all = df[FEATURES].to_numpy(float)
y_all = df['log_Price'].to_numpy(float)
```

```
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X_all, y_all, test_size=0.4, random_state=42)
X_val, X_te, y_val, y_te = train_test_split(X_tmp, y_tmp, test_size=0.5, random_state=42)
sc = StandardScaler().fit(X_tr)
X_tr, X_val, X_te = sc.transform(X_tr), sc.transform(X_val), sc.transform(X_te)
print(len(FEATURES), 'признаков:', X_tr.shape)
```

2.3.2. GD + MSE + L_1

Вариант 1 — обычный (суб)градиентный спуск по $F = Q_{MSE} + \lambda\|w\|_1$.

Вариант 2 — то же плюс принудительное зануление: после шага $w_j := 0$, если $|w_j| < \varepsilon_w$. Критерий: $\varepsilon_w = \eta\lambda$. **Обоснование:** за один шаг L_1-слагаемое сдвигает вес ровно на $\eta\lambda$ к нулю. Если $|w_j| < \eta\lambda$, следующий шаг «перепрыгнет» ноль, и вес будет колебаться около нуля, не становясь им. Это то же, что даёт проксимальный шаг (мягкий порог, см. теорию).

```
class GDZeroing(GD):
    """GD + зануление малых весов (кроме свободного члена) после каждого шага."""
    def __init__(self, lr, eps_w):
        super().__init__(lr=lr); self.eps_w = eps_w; self.name = 'GD + зануление'
    def step(self, w, g):
        w = super().step(w, g)
        small = np.abs(w) < self.eps_w
        small[0] = False
        w[small] = 0.0
        return w

LAM_GRID = [1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1, 10]
LR = 0.05          # шаг для стандартизованных данных (подберите по валидации, как в ЛР5)
EPS = 1e-6

def fit_l1(lam, zeroing, max_iter=5000):
    opt = GDZeroing(LR, eps_w=LR * lam) if zeroing else GD(lr=LR)
    loss = RegularizedLoss(MSELoss(), L1(lam))
    return LinearRegressionGD(loss, opt, eps=EPS, max_iter=max_iter, track_every=10 ** 9).fit(X_tr,
y_tr)

l1_results = {}
for zeroing in [False, True]:
    scores = {lam: rmse(y_val, fit_l1(lam, zeroing).predict(X_val)) for lam in LAM_GRID}
    best_lam = min(scores, key=scores.get)
    l1_results[zeroing] = (best_lam, fit_l1(best_lam, zeroing, max_iter=20000))
    print('зануление' if zeroing else 'без зануления', {k: round(v, 4) for k, v in scores.items()},
          '→ λ =', best_lam)
```

Наблюдение: при больших λ валидационная ошибка растёт — модель «съёживается» к константе. При $\lambda \geq \lambda_{\max} = \frac{2}{\ell}\|X^T(y - \bar{y})\|_\infty$ все веса нулевые (см. теорию).

2.3.3. Сравнение с `Lasso`

`Lasso` минимизирует $\frac{1}{2\ell}\|y - Xw\|_2^2 + \alpha\|w\|_1$. Наш функционал $\frac{1}{\ell}\|Xw - y\|_2^2 + \lambda\|w\|_1$ в два раза больше, поэтому $\alpha = \lambda/2$ даёт ту же точку минимума.

```
from sklearn.linear_model import Lasso, Ridge, LinearRegression

alphas = [1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1]
lasso_scores = {a: rmse(y_val, Lasso(alpha=a, max_iter=50000).fit(X_tr, y_tr).predict(X_val)) for a
in alphas}
best_alpha = min(lasso_scores, key=lasso_scores.get)
lasso = Lasso(alpha=best_alpha, max_iter=50000).fit(X_tr, y_tr)
print('Lasso α =', best_alpha)

coef = pd.DataFrame({
    'L1 GD':            l1_results[False][1].coef_,
    'L1 GD + зануление': l1_results[True][1].coef_,
    'Lasso':            lasso.coef_,
}, index=FEATURES)
quality = pd.Series({
    'L1 GD':            rmse(y_te, l1_results[False][1].predict(X_te)),
    'L1 GD + зануление': rmse(y_te, l1_results[True][1].predict(X_te)),
    'Lasso':            rmse(y_te, lasso.predict(X_te)),
}, name='RMSE test')
display(coef.round(4))
display(quality.round(4))

# Проверка реализации: Lasso с α = λ/2 должен дать почти те же веса, что наша модель с тем же λ
lam_z = l1_results[True][0]
same = Lasso(alpha=lam_z / 2, max_iter=50000).fit(X_tr, y_tr)
print('max |Δw| (своя L1 + зануление vs Lasso α=λ/2):',
      np.abs(same.coef_ - l1_results[True][1].coef_).max().round(4))

TOL = 1e-3
zeroed = {m: set(coef.index[coef[m].abs() < TOL]) for m in coef}
for m, s in zeroed.items():
    print(f'{m}: занулено/малых {len(s)} → {sorted(s)}')
```

2.3.4. Анализ отбора признаков

```
kept = {m: set(FEATURES) - s for m, s in zeroed.items()}
def jacc(a, b): return len(a & b) / max(len(a | b), 1)
display(pd.DataFrame([[jacc(kept[a], kept[b]) for b in kept] for a in kept],
                     index=list(kept), columns=list(kept)).round(2))
```

```
origin = {f: ('исходный' if f in base else 'предметный' if f in domain else
              'линеаризованный' if f in linear else 'dummy') for f in FEATURES}
display(pd.DataFrame({m: pd.Series({f: (f in kept[m]) for f in FEATURES}) for m in kept})
        .assign(тип=pd.Series(origin)).sort_values('тип'))
```

Корреляционная структура после отбора и качество модели на сокращённом пространстве:

```
sel = [f for f in FEATURES if f in kept['Lasso']]
idx = [FEATURES.index(f) for f in sel]
cmat = pd.DataFrame(X_tr[:, idx], columns=sel).corr()
plt.figure(figsize=(9, 7)); sns.heatmap(cmat, cmap='coolwarm', center=0); plt.title('Пирсон после
отбора'); plt.show()

full_corr = pd.DataFrame(X_tr, columns=FEATURES).corr().abs()
off = lambda c: c.where(~np.eye(len(c), dtype=bool)).stack()
print('max |r| между признаками: до отбора', off(full_corr).max().round(3), '| после',
off(cmat.abs()).max().round(3))

ols_full = LinearRegression().fit(X_tr, y_tr)
ols_sel = LinearRegression().fit(X_tr[:, idx], y_tr)
print('RMSE test: все признаки', round(rmse(y_te, ols_full.predict(X_te)), 4),
      '| отобранные', round(rmse(y_te, ols_sel.predict(X_te[:, idx])), 4))
```

Что обсудить: какие исходные и созданные признаки сохранились; из пар почти дублей (`BuildingArea` – `log_BA` – `AreaPerRoom`, `YearBuilt` – `Age`) L_1 обычно оставляет один; согласуются ли наборы (Жаккар); насколько упала максимальная взаимная корреляция; почти не изменилось качество при меньшем числе признаков — значит, отброшенные были избыточны.

2.4.1. Аналитическое МНК

```
def add_bias(X): return np.hstack([np.ones((len(X), 1)), X])

Xb_tr, Xb_val, Xb_te = add_bias(X_tr), add_bias(X_val), add_bias(X_te)
A = Xb_tr.T @ Xb_tr
print('κ(XᵀX) =', f'{np.linalg.cond(A):.3e}')
w_ols = np.linalg.solve(A, Xb_tr.T @ y_tr)       # решаем систему, а не обращаем матрицу
print('RMSE test OLS:', round(rmse(y_te, Xb_te @ w_ols), 4))
pd.Series(w_ols, index=['bias'] + FEATURES).round(4)
```

Если `solve` выдаёт `LinAlgError` (матрица вырождена), используйте `np.linalg.lstsq` или псевдообратную `np.linalg.pinv`, и объясните, какие признаки линейно зависимы. Огромное κ уже говорит о почти линейной зависимости (например, `Age` + `YearBuilt` ≈ год продажи).

2.4.2. Аналитический Ridge

$$w^* = (X^T X + \lambda P)^{-1} X^T y, \qquad P = \mathrm{diag}(0, 1, \ldots, 1),$$

нулём в P исключаем свободный член из штрафа.

```
P = np.eye(Xb_tr.shape[1]); P[0, 0] = 0.0
def ridge_closed(lam):
    return np.linalg.solve(A + lam * P, Xb_tr.T @ y_tr)

RIDGE_GRID = [1e-3, 1e-2, 1e-1, 1, 10, 100, 1000, 1e4]
ridge_val = {lam: rmse(y_val, Xb_val @ ridge_closed(lam)) for lam in RIDGE_GRID}
lam_star = min(ridge_val, key=ridge_val.get)
w_ridge = ridge_closed(lam_star)
print('λ* =', lam_star, '| κ(XᵀX+λP) =', f'{np.linalg.cond(A + lam_star * P):.3e}')
print('RMSE test Ridge:', round(rmse(y_te, Xb_te @ w_ridge), 4))
pd.DataFrame({'OLS': w_ols, 'Ridge': w_ridge}, index=['bias'] + FEATURES).round(4)
```

Сопоставьте: веса Ridge по модулю меньше («сжатие»), особенно у коррелированных признаков, где OLS давал большие веса разных знаков; $\|w\|_2$ уменьшилась; качество на тесте не хуже.

2.4.3. Сравнение с `Ridge` sklearn

`Ridge` минимизирует $\|y - Xw\|_2^2 + \alpha\|w\|_2^2$ без штрафа на свободный член — это ровно наша формула при $\alpha = \lambda$.

```
sk_val = {a: rmse(y_val, Ridge(alpha=a).fit(X_tr, y_tr).predict(X_val)) for a in RIDGE_GRID}
a_star = min(sk_val, key=sk_val.get)
sk_ridge = Ridge(alpha=a_star).fit(X_tr, y_tr)
print('α* sklearn =', a_star, '| λ* своё =', lam_star)
print('max |Δw| при одинаковом λ:',
      np.abs(Ridge(alpha=lam_star).fit(X_tr, y_tr).coef_ - w_ridge[1:]).max())
print('RMSE test: своё', round(rmse(y_te, Xb_te @ w_ridge), 4),
      '| sklearn', round(rmse(y_te, sk_ridge.predict(X_te)), 4))
```

Различие весов на уровне 10^{-10} — машинная точность. Если отличие заметное — проверьте, что свободный член не регуляризуется и данные одни и те же.

2.4.4. Влияние L_2 на градиентную оптимизацию

Берём функционалы из ЛР5 и **новый** метод — тот, которого не было в ЛР5 (если там были GD, Momentum, Adam — возьмите RMSProp). Для каждого функционала: $\lambda = 0$ и $\lambda > 0$ (подбор λ и шага по валидации).

```
NEW_OPT = lambda lr: RMSProp(lr=lr)
base_losses = {'MSE': MSELoss(), 'MAE': MAELoss(), 'Huber': HuberLoss(0.3), 'Log-cosh':
LogCoshLoss()}

def tune_l2(loss_name, lams, lrs=(1e-3, 1e-2, 1e-1), max_iter=3000):
    best = None
    for lam in lams:
        for lr in lrs:
            loss = RegularizedLoss(base_losses[loss_name], L2(lam))
            m = LinearRegressionGD(loss, NEW_OPT(lr), eps=EPS, max_iter=max_iter,
```

```
                                    track_every=10 ** 9).fit(X_tr, y_tr)
            s = rmse(y_val, m.predict(X_val))
            if np.isfinite(s) and (best is None or s < best[2]):
                best = (lam, lr, s)
    return best

runs = {}
for ln in base_losses:
    lam0, lr0, _ = tune_l2(ln, [0.0])
    lam1, lr1, _ = tune_l2(ln, [1e-3, 1e-2, 1e-1, 1])
    for tag, lam, lr in [('без L2', lam0, lr0), (f'L2, λ={lam1:g}', lam1, lr1)]:
        loss = RegularizedLoss(base_losses[ln], L2(lam))
        runs[(ln, tag)] = LinearRegressionGD(loss, NEW_OPT(lr), eps=EPS, max_iter=20000,
                                             track_every=100).fit(X_tr, y_tr)
```

```
for ln in base_losses:
    fig, axes = plt.subplots(1, 3, figsize=(16, 3.6))
    for (l, tag), m in runs.items():
        if l != ln:
            continue
        h = m.history
        axes[0].semilogy(h['grad_norm'], label=tag)
        axes[1].semilogy(h['kappa_iter'], h['kappa'], marker='.', label=tag)
        axes[2].semilogy(np.array(h['loss']) - min(h['loss']) + 1e-12, label=tag)
    axes[0].axhline(EPS, color='k', ls='--', label='ε')
    axes[0].set_title(f'{ln}: ‖∇F‖₂'); axes[1].set_title('κ(H_F)'); axes[2].set_title('F(w_k) − min
F')
    for ax in axes:
        ax.set_xlabel('k'); ax.legend(fontsize=8)
    plt.tight_layout(); plt.show()

pd.DataFrame([{'функционал': l, 'вариант': t, 'итераций': m.n_iter_, 'сошёлся': m.converged,
               'κ в конце': m.history['kappa'][-1], 'RMSE test': rmse(y_te, m.predict(X_te))}
              for (l, t), m in runs.items()]).round(4)
```

Что анализировать:

- **обусловленность:** $\kappa(H + \lambda P) = \frac{\lambda_{\max} + \lambda}{\lambda_{\min} + \lambda}$ (по нерегуляризованным направлениям) — малые собственные числа «поднимаются», κ падает;
- **сходимость:** при меньшем κ норма градиента падает быстрее и ровнее;
- **разные функционалы:** у Huber без L_2 гессиан может быть вырожден ($\kappa = \infty$, мало точек в зоне $|r| < \delta$) — L_2 делает его невырожденным по всем направлениям, кроме свободного члена; у MAE гессиана нет, L_2 добавляет только λP;
- **качество:** при подобранном λ RMSE на тесте не хуже, а веса устойчивее.

Чек-лист

- [ ] 2.1: производные L_1, L_2, точки недифференцируемости (TeX).
- [ ] 2.2: `Regularizer`, `L1`, `L2`, `RegularizedLoss` работают с любым `Loss`.
- [ ] 2.3.1: ≥ 3 предметных + ≥ 3 линеаризованных признака, два раза Пирсон, исходные сохранены.
- [ ] 2.3.2: L_1 без и с занулением, критерий зануления обоснован, λ по валидации.
- [ ] 2.3.3: `Lasso` ($\alpha = \lambda/2$), таблица коэффициентов, RMSE, занулённые признаки.
- [ ] 2.3.4: какие признаки сохранились, согласованность, корреляции после отбора, качество.
- [ ] 2.4.1–2.4.3: аналитические OLS и Ridge, сравнение со sklearn.
- [ ] 2.4.4: новый оптимизатор, без/с L_2 для каждого функционала, 3 графика, RMSE, выводы.